Catálogo y Diccionario Técnico de Funciones Utilizadas en el Proyecto Final KDD

Metadato	Detalle
Proyecto	Detección Automatizada de Fraude con Tarjetas de Crédito mediante Proceso KDD
Autor	Roberto Alejandro Chagra Martínez
Materia	Minería de Datos y Descubrimiento de Conocimiento (*Data Mining & Knowledge Discovery*)

1. Funciones Propias del Proyecto (Arquitectura Modular KDD)

Estas funciones fueron diseñadas específicamente para estructurar y automatizar las 7 etapas del proceso KDD dentro de `Proyecto_final_data_mining.py` y los módulos de `src/Entrega_Modular_de_Ingeniería_de_Software_en_Python/`.

Nombre de la función	Breve descripción de lo que hace	Sintaxis (cómo se escribe en el código)	Qué resultado generó en el proyecto
`obtener_ruta_dataset`	Verifica si el archivo CSV existe localmente en la carpeta del proyecto; si no existe, lo descarga automáticamente desde Kaggle.	`csv_path = obtener_ruta_dataset()`	Localizó y retornó la ruta local de `credit_card_fraud_detection_202` (`100,000` registros).
`cargar_y_seleccionar_datos`	Ejecuta las Etapas 1 y 2 KDD: carga el CSV, cuantifica el desbalance de clases, grafica tipologías de fraude y elimina columnas con *Data Leakage*.	`df_raw, df_sel = cargar_y_seleccionar_datos(csv_path, out_dir)`	Redujo el dataset de `43` a `35` columnas legítimas y exportó la gráfica `01_desbalance_y_tipologias.png`.
`limpiar_y_diagnosticar_outliers`	Ejecuta la Etapa 3 KDD: imputa los valores nulos de `browser`, verifica duplicados y calcula límites de Tukey ($1.5 \times \text{IQR}$) junto con el *Lift* de fraude.	`df_limpio, df_iqr = limpiar_y_diagnosticar_outliers(df_sel, out_dir)`	Dejó el dataset con `0` nulos conservand `100,000` filas y generó `diagnostico_outliers_iqr.csv` (demostrando un `Lift = 3.26x` en outliers).
`construir_variables_derivadas`	Construye 8 nuevas características financieras (velocidad relativa, Z-score a 30 días, nivel de seguridad física e índice multiseñal).	`df_feat = construir_variables_derivadas(df_limpio)`	Expandió el dataset con `8` variables derivadas; `composite_risk_index >=` concentró un `20.11%` de tasa real de fr (`9.79x` Lift).
`transformar_y_particionar`	Ejecuta la Etapa 4 KDD: aplica *One-Hot Encoding*, divide en Train/Test (80/20 estratificado), aplica `log1p` + `RobustScaler` y calcula la prueba χ^2.	`datos_etapa4 = transformar_y_particionar(df_limpio)`	Generó `X_train` (`80,000` × `91`) y `X_te` (`20,000` × `91`) escalados sin fuga y el ranking χ^2 encabezado por `high_risk_country` (`1,625.74`).
`ejecutar_mineria_datos`	Ejecuta la Etapa 5 KDD: entrena `IsolationForest`, filtra las 45	`datos_etapa5 = ejecutar_mineria_datos(datos_etapa4, out_dir)`	Generó el score de anomalía (`Lift = 3.75x`), balanceó `X_train` (`78,356` vs `23,506`) y entrenó Dummy, LogReg,

Nombre de la función	Breve descripción de lo que hace	Sintaxis (cómo se escribe en el código)	Qué resultado generó en el proyecto
	mejores variables, aplica `SMOTE` en entrenamiento y entrena los 5 clasificadores.		Decision Tree, Random Forest y Gradien Boosting.
`evaluar_e_interpretar_conocimiento`	Ejecuta las Etapas 6 y 7 KDD: grafica curvas ROC/PR, matrices de confusión, importancia Gini, árbol de reglas y valida las 5 reglas de negocio.	`df_res, df_reglas = evaluar_e_interpretar_conocimiento(datos_etapa4, datos_etapa5, out_dir)`	Exportó las gráficas `03` a `06`, `resumen_modelos_kdd.csv`, `reglas_decision_fraude.txt` y `reglas_negocio_validadas.csv` (has `33.83%` de precisión).

2. Funciones de Manipulación, Limpieza y Transformación de Datos (`pandas` y `numpy`)

Nombre de la función	Breve descripción de lo que hace	Sintaxis (cómo se escribe en el código)	Qué resultado generó en el proyecto	Por qué necesari utilizaci
`pd.read_csv()`	Lee un archivo de valores separados por comas (`.csv`) desde el disco y lo estructura en un `DataFrame` de Pandas.	`df_raw = pd.read_csv(csv_path)`	Cargó en memoria la matriz de `100,000` filas y `43` columnas transaccionales.	Es el pun entrada indispens para inge datos transacci bruto ha entorno
`Series.value_counts()`	Cuenta la frecuencia absoluta o relativa (`normalize=True`) de cada categoría única en una columna.	`df_raw["is_fraud"].value_counts(normalize=True) * 100`	Reveló que existen `97,945` operaciones legítimas (`97.945%`) y solo `2,055` fraudes (`2.055%`), y que *Card-Not-Present* suma `1,054` fraudes.	Permite diagnost cuantitat la severic desbalan clases an elegir la de mode
`DataFrame.drop()`	Elimina filas o columnas específicas de un `DataFrame` según sus nombres e índice de eje (`columns=...`).	`df = df_raw.drop(columns=cols_id + cols_leakage).copy()`	Eliminó `8` columnas (`6` identificadores y `2` variables post-evento), dejando `35` columnas válidas.	Es obliga la Etapa ((Selecció purgar at irrelevan variables provocar *Leakage*.
`DataFrame.isnull().sum()`	Detecta valores faltantes (`NaN`) por celda y suma el total de ausencias en cada columna.	`nulos = df.isnull().sum()`	Identificó exactamente `41,709` valores nulos concentrados exclusivamente en la columna `browser` y `0` en las demás.	Permite a integrida dataset y columna incomple de entre modelos matemát
`Series.fillna()`	Reemplaza los valores nulos (`NaN`) de una serie por un valor constante o estadístico especificado.	`df["browser"] = df["browser"].fillna("No_Aplica_Fisico")`	Sustituyó los `41,709` `NaN` (provenientes de compras en tienda `POS` y cajeros `ATM`) por la categoría `'No_Aplica_Fisico'`.	Los algor `scikit-` aceptan ` `NaN`; imp una cate semántic perder el de los da
`DataFrame.duplicated().sum()`	Verifica si existen filas idénticas repetidas en todas	`duplicados = df.duplicated().sum()`	Confirmó que existen `0` registros duplicados en las `100,000` transacciones.	Evita que observac repetidas

Nombre de la función	Breve descripción de lo que hace	Sintaxis (cómo se escribe en el código)	Qué resultado generó en el proyecto	Por qué necesari utilizaci
	las columnas y devuelve el conteo total.			los pesos entrenan se filtren simultán entre Tra
`Series.quantile()`	Calcula los cuantiles o percentiles estadísticos ($Q_1 = 0.25$, Mediana $= 0.50$, $Q_3 = 0.75$) de una variable numérica.	`q1, q2, q3 = df[col].quantile([0.25, 0.50, 0.75])`	Permitió calcular el rango intercuartílico ($\text{IQR} = Q_3 - Q_1$) y detectar `8,990` outliers en `transactions_last_1h` y `11,150` en `amount_last_1h`.	Provee u medida r paramétr robusta p identifica atípicos (mediante de Tukey IQR).
`DataFrame.groupby().agg()`	Agrupa registros según una variable categórica o discreta y calcula múltiples métricas agregadas simultáneamente.	`df.groupby("composite_risk_index")["is_fraud"].agg(["count", "mean"])`	Demostró que al subir `composite_risk_index` de `0` a `6`, la tasa media de fraude escala de `0.04%` a `19.19%`.	Esencial validar hi de negoc medir có cambia l probabili fraude en segment
`pd.get_dummies()`	Transforma variables categóricas de texto en columnas numéricas binarias (`0` y `1`) mediante *One-Hot Encoding*.	`pd.get_dummies(df, columns=cat_cols, drop_first=True, dtype=int)`	Convirtió las `8` variables categóricas nominales en variables *dummy*, ampliando la matriz a `92` columnas numéricas.	Los algor matemát (Regresió Logística Bosques, requieren entradas numérica `drop_fi` evita multicoli exacta.
`np.log1p()`	Calcula el logaritmo natural de $(1 + x)$ elemento por elemento: $\ln(1 + x)$.	`X_train[log_cols] = np.log1p(np.maximum(0, X_train[log_cols]))`	Comprimió la asimetría extrema de montos y distancias (ej. `transaction_amount` de `15,000` a `9.62`) manteniendo el `0.0` intacto.	Evita que extremos distorsio varianza tratar var con exce ceros sin $-\infty$.
`DataFrame.to_csv()`	Exporta un `DataFrame` de Pandas a un archivo `.csv` persistente en disco.	`df_res.to_csv(out_dir / "resumen_modelos_kdd.csv", index=False)`	Guardó las tablas de evaluación (`resumen_modelos_kdd.csv`, `reglas_negocio_validadas.csv` y `diagnostico_outliers_iqr.csv`).	Permite d evidencia auditable resultado cuantitat proyecto la memo tempora

3. Funciones de Partición, Escalamiento y Selección de Variables (`scikit-learn` — Etapa 4 KDD)

Nombre de la función	Breve descripción de lo que hace	Sintaxis (cómo se escribe en el código)	Qué resultado generó en el proyecto	Por qué es necesaria su utilización
`train_test_split()`	Divide matrices de datos en subconjuntos aleatorios de entrenamiento (`Train`) y prueba (`Test`), manteniendo la proporción de clases con `stratify=y`.	`train_test_split(X_orig, y, test_size=0.20, random_state=42, stratify=y)`	Separó `80,000` transacciones para entrenamiento (`1,644` fraudes, `2.055%`) y `20,000` para prueba (`411` fraudes, `2.055%`).	Sin `stratify=y`, en un dataset tan desbalanceado la partición aleatoria podría alterar la proporción de fraudes entre Train y Test.
`RobustScaler()` (`fit_transform` y `transform`)	Escala variables numéricas restando la mediana (Q_2) y dividiendo entre el rango intercuartílico ($\text{IQR} = Q_3 - Q_1$).	`scaler = RobustScaler()` `X_train[num] = scaler.fit_transform(X_train[num])` `X_test[num] = scaler.transform(X_test[num])`	Estandarizó las `26` variables numéricas en `X_train` y proyectó la misma transformación sobre `X_test`.	A diferencia de `StandardScaler` o `MinMaxScaler`, `RobustScaler` no se contamina con los *outliers* de fraude que decidimos conservar.
`chi2()`	Calcula el estadístico Chi-Cuadrado (χ^2) y el p-valor de dependencia entre cada característica no negativa/binaria y la clase `y`.	`chi_vals, p_vals = chi2(X_train[bin_cols], y_train)`	Identificó las `18` variables binarias más relevantes, lideradas por `high_risk_country` ($\chi^2 = 1,625.74, p \approx 0$).	Permite realizar reducción de dimensionalidad supervisada descartando variables *dummy* irrelevantes que solo agregan ruido.

4. Funciones de Minería de Datos: Anomalías, Balanceo y Clasificación (`scikit-learn` e `imbalanced-learn` — Etapa 5 KDD)

Nombre de la función	Breve descripción de lo que hace	Sintaxis (cómo se escribe en el código)	Qué resultado generó en el proyecto	Por qué es necesar utilización
`IsolationForest()`	Algoritmo no supervisado basado en árboles aleatorios que aísla observaciones atípicas en pocas particiones.	`iso = IsolationForest(n_estimators=150, contamination=0.035, random_state=42, n_jobs=-1)`	Detectó `665` transacciones anómalas en prueba con una tasa de fraude del `7.07%` frente a `1.88%` en las normales (`Lift = 3.75x`).	Cumple el objetivo **detección de valor atípicos (*outlier detection*)** en un espacio multidimensional si depender de etiqu previas.
`iso.score_samples()`	Calcula el puntaje continuo de normalidad/atipicidad de cada observación según la profundidad media en el bosque de aislamiento.	`X_train["iso_anomaly_score"] = -iso_forest.score_samples(X_train[iso_features])`	Generó la variable continua `iso_anomaly_score`, que se ubicó dentro del Top 10 de variables más importantes del modelo final.	Convierte el hallazg supervisado en una meta-característica numérica que enriq a los clasificadores supervisados.
`SMOTE()` y `fit_resample()`	Técnica de Sobremuestreo de Minorías Sintéticas que interpola nuevos ejemplos de la clase minoritaria usando vecinos cercanos (k-NN).	`smote = SMOTE(sampling_strategy=0.30, random_state=42)` `X_tr_sm, y_tr_sm = smote.fit_resample(X_train_sel, y_train)`	Incrementó los ejemplos de fraude en entrenamiento de `1,644` a `23,506` frente a `78,356` legítimos.	Resuelve el problem central del proyecto evita que la clase mayoritaria (`97.95%` enmascare a la minoritaria (`2.05%`) durante el aprendiz
`DummyClassifier()`	Clasificador base ingenuo que predice siempre la clase mayoritaria sin analizar las variables predictoras.	`DummyClassifier(strategy="most_frequent")`	Obtuvo `97.95%` de *Accuracy*, pero `0.00%` de *Recall* (`0` fraudes detectados de `411`) y `ROC-AUC = 0.5000`.	Sirve como línea ba (*baseline*) para demostrar empíricamente que alta exactitud (*Accu* no equivale a un bu modelo en desbalan
`LogisticRegression()`	Modelo lineal supervisado que estima la probabilidad de	`LogisticRegression(max_iter=500, C=1.0, random_state=42)`	Alcanzó `ROC-AUC = 0.8152` y detectó `178` fraudes (`Recall`	Proporciona un pun de referencia paramétrico rápido estable para contra

Nombre de la función	Breve descripción de lo que hace	Sintaxis (cómo se escribe en el código)	Qué resultado generó en el proyecto	Por qué es necesar utilización
	pertenencia a la clase positiva mediante la función sigmoide.		= 43.31%) en el conjunto de prueba.	contra modelos no lineales y de ensamb
`DecisionTreeClassifier()`	Algoritmo no lineal que divide recursivamente el espacio de variables mediante umbrales lógicos (`SI-ENTONCES`) minimizando la impureza Gini.	`DecisionTreeClassifier(max_depth=6, min_samples_leaf=50, random_state=42)`	Logró el mejor `F1-Score = 0.1997` (`Precision = 13.49%`, `Recall = 38.44%`, `ROC-AUC = 0.8103`) y permitió extraer reglas explícitas.	Es indispensable tan para clasificar como extraer **reglas de decisión comprens** por analistas human en la Etapa 7 KDD.
`RandomForestClassifier()`	Ensamble tipo *Bagging* que entrena cientos de árboles de decisión sobre muestras bootstrap y subconjuntos aleatorios de variables.	`RandomForestClassifier(n_estimators=200, max_depth=12, min_samples_leaf=15, class_weight="balanced_subsample", random_state=42)`	Logró la mayor sensibilidad operativa capturando **`220` de los `411` fraudes (`Recall = 53.53%`)** con `ROC-AUC = 0.8292`.	Reduce la varianza y sobreajuste (*overfitt* de un árbol individu provee el vector `feature_importan` (importancia Gini).
`GradientBoostingClassifier()`	Ensamble secuencial tipo *Boosting* donde cada nuevo árbol corrige los errores residuales de los árboles anteriores.	`GradientBoostingClassifier(n_estimators=150, max_depth=4, learning_rate=0.08, random_state=42)`	Obtuvo el mejor poder discriminante global del proyecto: **`ROC-AUC = 0.8379`** y **`PR-AUC = 0.1369`** (`6.68x` sobre el azar).	Especialmente eficaz para concentrarse e instancias difíciles (señales débiles de fraude) y maximizar área bajo la curva Precision-Recall.
`modelo.predict_proba()`	Devuelve la probabilidad continua estimada $[0.0, 1.0]$ de que cada transacción pertenezca a la clase de fraude (`1`).	`probs = modelo.predict_proba(X_test_sel)[:, 1]`	Generó los puntajes de riesgo continuo para las `20,000` transacciones de prueba.	Permite calibrar el umbral de corte ópt (`th` entre `0.15` y `0.5` maximizando el F_2-Score en lugar usar el umbral rígido `0.50`.

5. Funciones de Evaluación y Extracción de Conocimiento (`scikit-learn.metrics` y `tree` — Etapas 6 y 7 KDD)

Nombre de la función	Breve descripción de lo que hace	Sintaxis (cómo se escribe en el código)	Qué resultado generó en el proyecto	Por qué es necesaria utilización
`accuracy_score()`	Calcula la proporción de predicciones totales correctas $\frac{TP+TN}{TP+TN+FP+FN}$.	`acc = accuracy_score(y_test, preds)`	Mostró valores entre `87.86%` y `97.95%` en todos los modelos evaluados.	Se incluye para contras evidenciar su limitació metodológica frente al *PR-AUC* en bases desbalanceadas.
`precision_score()`	Calcula la pureza de las alertas de fraude: qué fracción de las transacciones marcadas como fraude realmente lo son ($\frac{TP}{TP+FP}$).	`prec = precision_score(y_test, preds, zero_division=0)`	Cuantificó una precisión de `13.49%` en Árbol de Decisión y `12.88%` en Gradient Boosting (`6.5x` mayor que la tasa base de `2.05%`).	Mide el costo operativ falsas alarmas (fricción clientes legítimos cuya son alertadas).
`recall_score()`	Calcula la sensibilidad o exhaustividad: qué fracción del total de fraudes reales logró atrapar el modelo ($\frac{TP}{TP+FN}$).	`rec = recall_score(y_test, preds, zero_division=0)`	Demostró que Random Forest atrapó el **`53.53%`** (`220` fraudes) y Regresión Logística el **`43.31%`** (`178` fraudes), frente al `0%` del Baseline.	Es la métrica crítica de en prevención de frau cada Falso Negativo re pérdida monetaria dire el banco.
`f1_score()`	Calcula la media armónica entre *Precision* y *Recall*: $2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$.	`f1 = f1_score(y_test, preds, zero_division=0)`	Posicionó al Árbol de Decisión (`0.1997`) y Gradient Boosting (`0.1956`) como los modelos más equilibrados.	Resume en un solo ind balance entre atrapar f no disparar falsas alarr excesivas.
`roc_auc_score()` y `roc_curve()`	Calculan las coordenadas y el área bajo la curva	`roc_auc = roc_auc_score(y_test, probs)`	Confirmó un desempeño sobresaliente en todos los modelos	Evalúa la capacidad glo ordenamiento del mod

Nombre de la función	Breve descripción de lo que hace	Sintaxis (cómo se escribe en el código)	Qué resultado generó en el proyecto	Por qué es necesaria su utilización
	Característica Operativa del Receptor (TPR vs. FPR en todos los umbrales).	`fpr, tpr, _ = roc_curve(y_test, probs)`	con SMOTE (`0.8103` a **`0.8379`** en Gradient Boosting).	independientemente d de corte elegido.
`average_precision_score()` y `precision_recall_curve()`	Calculan la curva y el área bajo la curva Precision-Recall (`PR-AUC`), enfocada exclusivamente en la clase minoritaria.	`pr_auc = average_precision_score(y_test, probs)` `prec_c, rec_c, _ = precision_recall_curve(y_test, probs)`	Elevó el `PR-AUC` desde `0.0205` (azar) hasta **`0.1369`** en Gradient Boosting y `0.1109` en Árbol de Decisión.	Es el estándar científic estricto para evaluar clasificadores bajo des severo, pues no se infl alto número de Verdac Negativos (`TN`).
`confusion_matrix()`	Construye la matriz 2×2 con el conteo exacto de Verdaderos Negativos (`TN`), Falsos Positivos (`FP`), Falsos Negativos (`FN`) y Verdaderos Positivos (`TP`).	`cm = confusion_matrix(y_test, preds)` `tn, fp, fn, tp = cm.ravel()`	Desglosó exactamente cuántos fraudes de los `411` de prueba fueron capturados (`TP`) y cuántos omitidos (`FN`) por cada algoritmo.	Permite auditar con transparencia absoluta aciertos y errores cont cada modelo.
`export_text()`	Extrae la estructura lógica de un `DecisionTreeClassifier` entrenado en formato de reglas de texto jerárquicas (`	--- variable <= umbral`).	`reglas_texto = export_text(dt_interpretable, feature_names=features_reglas, decimals=2)`	Generó el archivo `reglas_decision_fr` con los puntos de cort en unidades originales (`distance_from_home` `544.85`, `transactions_last_` `2.50`, etc.).
`plot_tree()`	Renderiza gráficamente el diagrama de nodos, ramas, umbrales y proporciones de clase de un árbol de decisión.	`plot_tree(dt_plot, feature_names=..., class_names=["Legítima", "FRAUDE"], filled=True, proportion=True)`	Generó el diagrama visual `06_arbol_reglas_decision.png` de profundidad 3 coloreado por pureza de clase.	Facilita la interpretació inmediata en la Etapa proceso KDD para tom de decisiones no técni

6. Funciones de Visualización Científica (`matplotlib` y `seaborn`)

Nombre de la función	Breve descripción de lo que hace	Sintaxis (cómo se escribe en el código)	Qué resultado generó en el proyecto	Por qué es necesaria su utilización
`plt.subplots()`	Crea una figura y una cuadrícula de subgráficos (`axes`) para organizar múltiples visualizaciones comparativas en un mismo lienzo.	`fig, axes = plt.subplots(1, 2, figsize=(14, 5))`	Estructuró los paneles dobles y cuádruples de las 6 figuras del proyecto.	Permite comparar lado a lado distribuciones, curvas ROC vs. PR y las 4 matrices de confusión sin saturar el reporte.
`sns.barplot()` y `ax.bar()`	Generan gráficos de barras horizontales o verticales para comparar magnitudes entre categorías o rankings de variables.	`sns.barplot(x=imp_series.values, y=imp_series.index, palette="Blues_r", ax=axes[0])`	Visualizó el desbalance de clases, las modalidades de fraude, la tasa de fraude por nivel de riesgo y el Top 15 de importancia de variables.	Comunica visualmente las jerarquías de riesgo y qué atributos tienen mayor peso predictivo.
`sns.kdeplot()`	Estima y grafica la función de densidad de probabilidad continua (*Kernel Density Estimate*) de una variable numérica separada por clase.	`sns.kdeplot(data=df_plot, x="Score Anomalía", hue="Clase", common_norm=False, fill=True)`	Mostró el desplazamiento hacia la derecha del puntaje `IsolationForest` en las transacciones fraudulentas respecto a las legítimas.	Con `common_norm=False`, permite comparar la forma de dos distribuciones aunque una clase tenga `97,945` datos y la otra `2,055`.

Nombre de la función	Breve descripción de lo que hace	Sintaxis (cómo se escribe en el código)	Qué resultado generó en el proyecto	Por qué es necesaria su utilización
`sns.heatmap()`	Representa una matriz bidimensional de números como un mapa de calor codificado por intensidad de color y anotaciones exactas.	`sns.heatmap(cm, annot=True, fmt="d", cmap="YlGnBu", ax=ax)`	Renderizó las 4 matrices de confusión en `05_matrices_confusion.png` con conteos enteros legibles.	Facilita la inspección visual inmediata de los cuadrantes de acierto (`TP`, `TN`) y error (`FP`, `FN`).
`fig.savefig()`	Exporta la figura activa en memoria hacia un archivo de imagen de alta resolución (`.png`).	`fig.savefig(out_dir / "04_comparativa_modelos_curvas_roc_pr.png", bbox_inches="tight")`	Guardó las 6 láminas analíticas en la carpeta `resultados_kdd/` con recorte limpio de márgenes.	Permite reutilizar las gráficas tanto en los Notebooks modulares como en reportes ejecutivos o presentaciones.